CONFIDENTIAL — ENGINEERING DIRECTIVE

KensGames.com

AI Improvement Directive & Manifold Tightening Protocol

ButterflyFX™ Engine Optimization Playbook

Author: **Kenneth Bingham**

Version 1.0 — April 2026

Kaysville, Utah

"Store Geometry — Not Bytes"

Field	Value
Document ID	KG-DIRECTIVE-2026-001
Classification	Internal Engineering Directive — AI-Readable
Author	Kenneth Bingham — Sole Creator, KensGames.com & ButterflyFX™
Version	1.0 — Initial Release
Date	20 April 2026
Scope	6 Site Improvement Directives + 8 Manifold Tightening Protocols
Audience	Any AI assistant collaborating with Kenneth on KensGames.com

Section 1: Executive Summary

Purpose. This document is a complete, AI-readable directive pack. Any AI assistant that ingests this document should be able to help Kenneth Bingham improve KensGames.com and harden the ButterflyFX™ manifold substrate architecture with precision, without ambiguity, and without violating the foundational principles of the project.

Context. KensGames.com is a paradigm manifesto disguised as an arcade. Five playable browser-based games — Fast Track, Alien Space Attack (Starfighter), BrickBreaker 3D, 4D TicTacToe, and Assemble — are built on a shared manifold substrate comprising **185 production modules**, **zero event buses**, and **formal complexity proofs**. The site simultaneously serves two audiences: casual arcade players who want to click and play, and technical readers who want to understand the substrate architecture that makes it possible.

Scope. This directive addresses:

- **Six site-level improvement directives** (Section 2) covering SEO routing, mobile responsiveness, casual visitor fast-paths, social proof, art store prominence, and privacy-preserving analytics.
- **Eight manifold-tightening protocols** (Section 3) covering substrate isolation, lens composition hardening, coordinate density, zero-coupling enforcement, formal proof validation, shared layer audits, shader alignment, and integration readiness for upcoming titles.

> ❗ **Guiding Philosophy**
>
> Every change must reinforce the manifold thesis. Nothing decorative. Nothing that introduces coupling. The architecture IS the product. If a proposed change cannot be expressed as a coordinate on the manifold surface, it does not belong.

Architecture Snapshot

Metric	Value
Total Production Modules	185
Shared Cross-Title Layer	46 modules (368 KB)
Game Substrates	5 (FastTrack: 96, ASA: 33, BB3D: 7, T4D: 1, Assemble: 2)
Event Buses	0
External Dependencies (npm)	0
Rendering Pipeline	Hand-written WebGL2 + GLSL
Coupling Reduction (Theorem 1)	92× at N=185
Upcoming Titles (Q3–Q4 2026)	4

Section 2: Site Improvement Directives

Six directives addressing user experience, discoverability, and brand surface area. Each directive includes the problem statement, precise AI implementation rules, and a validation checklist.

Directive 2.1 — Deep-Link URL Routing for SEO

Problem Statement

The single-page architecture means all content lives under one URL. Search engines see one page. Social shares link to the top of the page, not to specific sections. The formal proofs, engine specifications, and individual game titles cannot be directly linked, shared, or independently indexed by search crawlers.

AI Implementation Rules

Routing Mechanism: Implement History API-based routing using `pushState` / `replaceState`. Do NOT use hash-based (`#`) routing — hash fragments are not sent to servers and are treated as same-page by search engines.

Canonical Route Map:

Route	Target Section	Meta Title
`/games`	Game cards section	Free Arcade Games | KensGames
`/games/fasttrack`	Fast Track game card	Fast Track — Free Racing Game | KensGames
`/games/starfighter`	Alien Space Attack / Starfighter card	Starfighter — Free Space Shooter | KensGames
`/games/brickbreaker`	BrickBreaker 3D game card	BrickBreaker 3D — Free Brick Game | KensGames
`/games/tictactoe`	4D TicTacToe game card	4D TicTacToe — Free Strategy Game | KensGames
`/games/assemble`	Assemble game card	Assemble — Free Puzzle Game | KensGames
`/engine`	“The Engine” section	ButterflyFX Engine | KensGames
`/manifold`	“Under the Hood” formal proofs	Manifold Substrate — Formal Proofs | KensGames
`/about`	Creator bio section	About Kenneth Bingham | KensGames
`/coming-soon`	Upcoming titles	Coming Soon — New Games | KensGames
`/terms`	Terms of Service	Terms of Service | KensGames

Per-Route Metadata Requirements: Each route MUST have its own:

- Unique `<title>` tag (dynamically updated via `document.title`)
- Unique `<meta name="description">`

- Unique Open Graph tags: `og:title`, `og:description`, `og:image`
- Unique Twitter Card tags: `twitter:card`, `twitter:title`, `twitter:description`
- `<link rel="canonical">` pointing to the canonical URL for that route

Sitemap & Crawlability:

- Generate a dynamic `sitemap.xml` at `/sitemap.xml` listing all routes with `lastmod` dates
- Include `<link rel="canonical">` on every route to prevent duplicate content issues
- Pre-render critical routes for search engine crawlers using a lightweight pre-rendering approach (e.g., Puppeteer snapshot at build time) — NOT a full SSR framework, which would violate the zero-dependency principle

404 Handling: Unknown routes redirect to the main page with a toast notification: `"Route not found. Returning to base."` — functional, no emoji, no “Oops!”

Server-Side Configuration (Nginx):

```
location / {     try_files $uri $uri/ /index.html; }
```

Validation Checklist

☐	**Criterion**
☐	Each route renders the correct section in viewport
☐	Browser back/forward buttons work correctly across all routes
☐	Direct URL entry (cold navigation) loads the correct section
☐	Social share preview shows route-specific metadata (test via Facebook Debugger, Twitter Card Validator)
☐	Google Search Console shows all routes indexed within 30 days
☐	`sitemap.xml` is valid, accessible, and submitted to Search Console
☐	Canonical tags resolve correctly — no

☐	Criterion
	duplicate content warnings

Directive 2.2 — Mobile Responsiveness for Dense Technical Sections

Problem Statement

The formal proofs section, module count tables, and substrate architecture diagrams contain dense mathematical notation and wide tables that may overflow on mobile screens (< 768px). The WebGL2 gyroid background is GPU-intensive and may cause thermal throttling on low-tier mobile devices.

AI Implementation Rules

Mathematical Notation & Theorems:

- Wrap in horizontally scrollable containers with subtle scroll indicators (gradient fade on right edge)
- Scale font size using `clamp()`: minimum 14px, preferred 1.1vw, maximum 18px
- Break long expressions at operator boundaries (≤, →, ×) with CSS `word-break` rules

Module Count Tables:

- Convert to responsive card layout below 600px breakpoint
- Each row becomes a card: game title as card header, metrics as labeled key-value pairs
- Maintain neon accent color coding per game title

Substrate Architecture Diagrams:

- Implement pinch-to-zoom on mobile for any diagram or schematic
- Provide a “tap to expand” full-screen overlay for complex diagrams

WebGL2 Background (Gyroid Ray-March) — Progressive Degradation:

GPU Tier	Detection Method	Action
High (Desktop GPU)	WebGL renderer string contains “NVIDIA”, “Radeon”, etc.	Full quality — no changes
Mid (Flagship mobile)	Adreno 7xx, Apple GPU, Mali-G7x	Reduce ray-march to 48 steps, resolution at 0.75×
Low (Budget mobile)	Adreno 5xx/6xx, Mali-G5x or below	Reduce to 32 steps, resolution at 0.5× canvas
Fallback	WebGL2 unavailable or frame time > 33ms	Static CSS gradient mimicking gyroid palette

> ❗ **Critical Rule**
>
> Never disable the background entirely — it IS the thesis. Degrade gracefully. The gyroid is not decoration; it is the manifold made visible.

Touch Targets: Ensure all interactive elements (game launch buttons, nav links, social icons) meet the 44×44px minimum tap target size per WCAG guidelines.

Test Breakpoints: 320px, 375px, 414px, 768px, 1024px, 1440px, 1920px.

Performance Budget

Metric	Target	Notes
First Contentful Paint (mobile 4G)	< 2.5s	Measured via Lighthouse
Largest Contentful Paint	< 4.0s	Critical for Core Web Vitals
Cumulative Layout Shift	< 0.1	Reserve space for async content
Total JavaScript Bundle	≤ 5 MB	Current 4.78 MB acceptable for WebGL app; tree-shake dead code

Directive 2.3 — Casual Visitor Fast-Path (“Just Play” Mode)

Problem Statement

First-time visitors encounter the mathematical thesis before seeing a game. The manifold narrative is essential for technical audiences, but casual gamers may bounce before discovering the arcade section.

AI Implementation Rules

Floating “PLAY NOW” Button:

- Fixed position, bottom-right corner, `z-index` above all content
- Pill-shaped, neon cyan glow (`#00ffff`), pulses gently on idle, solid on hover
- Label: **PLAY NOW ▶**
- Clicking scrolls smoothly to the game cards section (the “■ NOW PLAYING ■” divider)
- Appears after 3 seconds on page; disappears once user scrolls past the game cards section

Secondary Micro-CTA: Add a small “↓ Skip to Games” text link below the thesis paragraph in the hero section. Styled subtly (smaller font, muted color) so it does not compete with the primary narrative.

First-Visit Welcome Modal (Optional):

- Triggered by absence of a `kg_visited` cookie
- Copy: “Welcome to KensGames! Choose your path:”
- Two options: **“Play Games”** (scrolls to arcade) and **“Explore the Engine”** (continues reading)
- Show only once. Set cookie on dismissal. No dark patterns. No forced interaction.

> 💡 **Design Principle**
>
> DO NOT hide or reduce the manifold thesis section. That content is the founder's core message. The fast-path is an *addition*, not a replacement. The thesis stays exactly where it is.

Directive 2.4 — Social Proof Integration

Problem Statement

No visible player counts, reviews, community testimonials, or activity indicators. The portal feels like a museum rather than a living arcade. New visitors have no social signal that others are playing.

AI Implementation Rules

Header — Live Player Counter:

- Format: 🟢 **[N] Players Online** — green dot pulses when count > 0
- Source: WebSocket connection count from the existing server
- If zero players online, display: **"Be the first to play today!"**

Per-Game Card Metrics:

Metric	Source	Display Format
Total Games Played	Server-side cumulative counter	`Total Games Played: 1,247`
High Score	Leaderboard data	`High Score: 98,400 by [Avatar]`
Last Played	Server-side timestamp	`Last Played: 2 minutes ago` (relative time)

Community Section (insert between game cards and Engine section):

- Rotating testimonials/quotes from players (seed with real feedback from Substack/social channels)
- Mini leaderboard widget: top 5 players across all games

- “Join the Community” links to Discord and/or Substack

> ⚠ **Integrity Rule**
>
> All metrics MUST be real data — never fake numbers. If the platform is early-stage with low traffic, display real numbers honestly. Authenticity > vanity metrics. This is non-negotiable.

Directive 2.5 — Art Store Prominence

Problem Statement

The Shopify/Printful art store appears in meta descriptions and footer but is not prominently visible in the main content flow. As a revenue channel, it deserves deliberate surface area within the site experience.

AI Implementation Rules

Navigation: Add an “Art Store” link in the top nav bar, positioned after “Games” and before “About.”

Dedicated Section:

- Title: ■ **ART STORE** ■ (matching the arcade divider aesthetic)
- Position: between “Coming Soon” and “About” sections
- Display 3–4 featured product cards with images, titles, and prices
- Each card links to the Shopify store in a new tab (`target="_blank"`)
- CTA button: **“Browse Full Collection →”**

Creator Bio Integration: Add a subtle “Art Store” badge or link on the creator bio section — Kenneth is the artist, so this connection is natural and authentic.

Boundary Rule: Do NOT embed the full Shopify store. Maintain separation between the game portal and commerce. External links in new tabs are the correct pattern.

Directive 2.6 — Privacy-Preserving Analytics

Problem Statement

“No ads. No tracking.” is a core brand value. However, some analytics are needed to understand user behavior and validate the improvements in this directive. The solution must respect the ethical positioning absolutely.

AI Implementation Rules

Recommended Solutions (in order of preference):

1. **Custom minimal analytics via the existing WebSocket server** — track page section views, game launches, and session duration server-side without any client-side tracking library
2. **Plausible Analytics (self-hosted)** — lightweight (< 1 KB script), cookie-free, GDPR-compliant
3. **Umami (self-hosted)** — open source, privacy-focused

Permitted Metrics (exhaustive list — nothing else):

Metric	Purpose	PII Risk
Page section scroll depth	Understand which sections get read	None
Game launch counts per title	Measure game popularity	None
Session duration	Engagement quality	None
Referrer domain	Traffic source attribution	None
Device category (mobile/tablet/desktop)	Responsive design validation	None
Browser + WebGL capability tier	Progressive enhancement decisions	None

Prohibited: IP addresses, user agents for fingerprinting, individual user journeys, personal data, third-party tracking scripts of any kind.

Footer Disclosure: Display a brief, honest note: *“We track anonymous usage statistics to improve your experience. No personal data is collected. No cookies are used.”*

Admin Dashboard: Make analytics data available in a simple, password-protected admin view for Kenneth.

Section 3: Manifold Tightening Protocols

Eight protocols for hardening the ButterflyFX™ manifold substrate. These are engineering-grade specifications, not suggestions. Each protocol includes an objective, precise implementation rules, and measurable acceptance criteria.

Protocol 3.1 — Substrate Isolation Verification

Objective

Mathematically guarantee that no substrate can access, modify, or depend on another substrate’s internal state. The current O(N) coupling claim must be empirically verifiable at build time with zero tolerance for violations.

AI Implementation Rules

Build-Time Static Analysis Pass:

```
{   "substrate": "fasttrack",   "internalRefs": 89,   "externalRefs": 7,
"sharedLayerRefs": 7,   "violations": [] }
```

Substrate Boundary Manifest (`substrates.json`):

```
{   "substrates": [     { "name": "shared",           "moduleCount": 46,
"boundary": "closed" },     { "name": "fasttrack",        "moduleCount": 96,
"boundary": "closed" },     { "name": "alienspaceattack", "moduleCount": 33,
"boundary": "closed" },     { "name": "brickbreaker",     "moduleCount":  7,
"boundary": "closed" },     { "name": "tictactoe",        "moduleCount":  1,
"boundary": "closed" },     { "name": "assemble",         "moduleCount":  2,
"boundary": "closed" }   ],   "legalCrossRefs": "game -> shared (read only)",
"illegalCrossRefs": "game -> game, shared -> game" }
```

Communication Rule: The ONLY legal cross-substrate communication is reading from the shared substrate's coordinate surface. No game substrate may write to the shared layer. No game substrate may reference another game substrate.

Protocol 3.2 — Lens Composition Hardening

Objective

Ensure that lenses (the read-only views into the substrate) are truly composable, side-effect-free, and cannot introduce hidden coupling.

Lens Properties — All Four Must Hold

Property	Definition	Enforcement
Pure	`lens(substrate, coordinate) → value`. Same inputs always produce same output. No side effects.	Unit tests with deterministic inputs; no I/O, no `Date.now()`, no `Math.random()` inside lenses
Read-only	A lens MUST NOT mutate the substrate.	Enforce with `Object.freeze()` on substrate references passed to lenses, or use a `Proxy` that throws on `set` operations
Composable	`lens_A(lens_B(substrate, coord_B), coord_A)` must equal the direct computation. Composition must be associative.	Property-based tests verifying associativity across lens chains
Bounded	Each lens must declare its coordinate domain — the set of coordinates it reads. No lens may read outside its declared domain.	LensRegistry + Proxy-based domain verification in debug mode

LensRegistry Specification

- Registers all lenses with their declared coordinate domains

- **Debug mode:** Wraps each lens in a `Proxy` that logs actual coordinate access; compares declared vs. actual access; flags domain violations
- **Production mode:** Strips the `Proxy` for zero overhead

Lens Composition Operator:

```
function composeLens(lensA, lensB) {   const composed = (substrate) =>
lensA(lensB(substrate));   composed.domain = [...new
Set([...lensA.domain, ...lensB.domain])];   return composed; }
```

Documentation Standard: Every lens must include JSDoc with `@domain`, `@pure true`, and `@returns` annotations.

Protocol 3.3 — Coordinate Density Optimization

Objective

Maximize information density at every point on the manifold surface. Reduce wasted coordinates. Ensure every game state maps efficiently to the geometric substrate.

AI Implementation Rules

Coordinate Audit (per substrate):

```
{   "substrate": "fasttrack",   "coordinates_total": 142,
"coordinates_active": 128,   "coordinates_dead": 14,   "density_ratio":
0.90 }
```

Dead Coordinate Elimination:

- Remove coordinates with zero lens readers — this is the geometric equivalent of dead code elimination
- Run after every build to prevent coordinate sprawl

Coordinate Packing for Sparse Values:

```
// Before: three separate coordinates player.status.isAlive    // boolean
player.status.isShielded // boolean player.status.weaponTier // 0-3  //
After: packed into one coordinate using bit-field encoding
player.status.packed = (isAlive << 0) | (isShielded << 1) | (weaponTier << 2)
// Transparent unpacking via lenses: function lens_isAlive(substrate)
{   return (substrate.player.status.packed >> 0) & 1; } function
lens_isShielded(substrate) {   return (substrate.player.status.packed >> 1) &
1; } function lens_weaponTier(substrate) {   return
(substrate.player.status.packed >> 2) & 3; }
```

Coordinate Naming Convention:

Rule	Format	Example
Hierarchy	`{domain}.{entity}.{property}[.{subproperty}]`	`physics.ball.velocity.x`
Domains	Fixed set: `player`, `game`, `physics`, `ui`, `network`, `audio`, `input`	`player.identity.avatar`
Entities	Lowercase singular nouns	`ball`, `paddle`, `score`, `state`
Rejected	Flat names that lose geometric structure	`playerScore` ❌, `isGameOver` ❌

Protocol 3.4 — Zero-Coupling Enforcement at the Event Layer

Objective

The site claims zero event buses. Verify this claim continuously and prevent regression. Define precisely what constitutes an “event bus” so detection is unambiguous.

Event Bus Definition (for Detection)

Pattern	Classification
Object implementing `subscribe(topic, callback)` + `publish(topic, data)`	**VIOLATION**
Global `EventEmitter`, `EventTarget`, or custom pub/sub	**VIOLATION**
Shared mutable array of callbacks indexed by string keys	**VIOLATION**
DOM `CustomEvent` dispatched on `document` or `window` for cross-module communication	**VIOLATION**
DOM events on actual DOM elements for UI interaction (`click`, `keydown`, etc.)	**PERMITTED** (whitelisted)

Build-Time Scanner Rules

- Grep all source files for: `addEventListener` on `document`/`window` with custom event names, `dispatchEvent` with `CustomEvent`, any class with `emit` + `on` + `subscribe` methods, any module exporting a shared callback registry
- Fail build on detection with specific `file:line` references

The Manifold Alternative: Coordinate-Mediated Communication (CMC)

Coordinate-Mediated Communication (CMC)

Instead of Module A emitting `"playerScored"` for Module B to hear:

1. Module A **writes** to substrate coordinate `game.score.current`
2. Module B's lens **reads** that coordinate on the next frame

All inter-module data flow MUST go through CMC. No exceptions. This is the manifold-native communication primitive.

Protocol 3.5 — Formal Proof Hardening (Theorems 1–3)

Objective

Strengthen the three published formal proofs with empirical validation, edge case analysis, and extensibility projections for the four upcoming game titles.

Theorem 1: Coupling Complexity — $O(N^2) \rightarrow O(N)$

Current Claim: $O(N^2)$ conventional coupling reduces to $O(N)$ manifold coupling, yielding a 92× reduction at N=185.

Hardening Steps:

4. Run the substrate isolation scanner (Protocol 3.1) and count actual edges in the dependency graph
5. Compare actual edge count **E** to:
    - Theoretical maximum (conventional): N×(N-1)/2 = **16,930** edges
    - Theoretical manifold target: N = **185** edges
6. If actual E > N, investigate each surplus edge — classify as legitimate shared-layer read or true coupling violation
7. Project for N=250 (after adding 4 new games with ~65 estimated new modules):
    - Conventional: 31,125 edges
    - Manifold: ~250 edges
    - Ratio improves from 92× to **125×**
8. Publish the projection: *"Scaling Proof: as KensGames grows, the architecture advantage accelerates."*

Theorem 2: Runtime Coordination Cost — O(N·k·m) → O(N·d)

Current Claim: O(N·k·m) brokered coordination reduces to O(N·d) manifold coordination.

Hardening Steps:

9. Instrument the game loop to measure actual per-frame coordination cost in microseconds
10. Log per frame: `frame_number`, `modules_active`, `coordinate_reads`, `total_coordination_µs`
11. Plot `coordination_µs` vs. `modules_active` — the relationship should be linear, confirming O(N·d)
12. Compare against a simulated event-bus architecture with the same module count — measure the overhead of subscribe/dispatch/callback resolution

13. Publish empirical results alongside the theoretical proof: *"Theorem 2 — Empirical Validation"* section on the site

Theorem 3: Grounded Inference / Hallucination Bound

Current Claim: $P(Q) \leq \rho(p_Q) / \rho_{max}$ — confidence bounded by local coordinate density.

Current Honest Disclaimer: *"The embedding function ε is an open engineering problem."*

Hardening Steps:

14. **Define a concrete embedding function** for the gaming domain: ε maps a natural-language query Q to a manifold coordinate p_Q by matching query tokens to coordinate path segments
15. **Example:** Q = "What is the player's score?" → tokenize → match "player" + "score" → p_Q = player.score.current → $\rho(p_Q)$ = (lenses reading this coordinate) / (max lenses on any coordinate)
16. **Implement a proof-of-concept query resolver:**
    - Takes a natural language question about game state
    - Maps it to a manifold coordinate via ε
    - Returns the value at that coordinate with confidence = $\rho(p_Q) / \rho_{max}$
    - If confidence < 0.3, returns: *"Low confidence — coordinate sparsely populated"* instead of hallucinating
17. Frame as **"Theorem 3 — Engineering Progress"** rather than claiming it is solved
18. The existence of a working ε for the gaming domain, even a simple one, is publishable and differentiating

> 💡 **Intellectual Honesty**

> Theorem 3 is the most ambitious claim. Frame all progress with the same rigor and honesty that the current disclaimer demonstrates. A working proof-of-concept with stated limitations is far more credible than an overclaimed solution.

Protocol 3.6 — Shared Layer Efficiency Audit

Objective

The shared cross-title layer (368 KB, 46 modules) is the backbone of the 5× leverage claim. Ensure it remains lean, necessary, and correctly bounded. Every byte in the shared layer must justify its presence.

AI Implementation Rules

Per-Module Audit Criteria:

Criterion	Question	Action if Failed
Usage count	How many game substrates import this module?	If only 1 game imports it, move to that game's substrate
Size justification	Is the module's size proportional to its usage?	A 50 KB module used by 1 game is waste — extract
Interface stability	Does the module's public API change frequently?	Unstable shared APIs are coupling vectors — freeze interface
Dependency direction	Does this shared module import from any game substrate?	VIOLATION — shared modules must never import from games

Shared Layer Health Report Output:

Category	Criteria	Action
Modules used by 5/5 games	True foundation	Keep in shared layer — protect and optimize
Modules used by 3–4	Strong shared candidates	Keep in shared layer

Category	Criteria	Action
games		
Modules used by 1–2 games	Candidates for extraction	Relocate to consuming game substrate(s)
Modules with zero imports	Dead shared code	Remove immediately

Target: The shared layer should contain *only* modules used by ≥ 3 game substrates. Everything else should be relocated or removed.

Protocol 3.7 — WebGL2 Shader Substrate Alignment

Objective

The hand-written GLSL shaders are a core differentiator. Ensure they follow the same substrate isolation principles as the JavaScript layer. Shaders are substrate modules; they must be documented and governed accordingly.

AI Implementation Rules

Shader-as-Substrate Mapping:

- Each shader program (vertex + fragment pair) is treated as a substrate module
- Shader **uniform inputs** = coordinate reads
- Shader **outputs** (`gl_FragColor`, `gl_Position`, varyings) = coordinate writes
- No shader may read uniforms from another shader's domain

Shader Manifest (`shaders.json`):

```
{   "shaders": [     {       "name": "gyroid_background",       "type":
"ray_march",       "uniforms_read": ["u_time", "u_resolution", "u_camera"],
"outputs": ["gl_FragColor"],       "domain": "background",
"estimated_gpu_cost": "high",       "mobile_adaptable": true     },     {
"name": "game_sprite",       "type": "textured_quad",       "uniforms_read":
["u_modelMatrix", "u_texture", "u_opacity"],       "outputs":
["gl_FragColor", "gl_Position"],       "domain": "game_rendering",
"estimated_gpu_cost": "low",       "mobile_adaptable": false     }   ] }
```

Gyroid Background — Dynamic Quality:

- Parameterize the ray-march step count as a uniform (`u_stepCount`) so it can be reduced on mobile without a separate shader file
- Add a resolution scale uniform (`u_resolutionScale`) for dynamic quality adjustment
- Implement frame time monitoring: if average frame time exceeds 16.67ms (60fps target), automatically reduce quality one tier; if it recovers, restore quality after a cooldown period

Protocol 3.8 — Coming Soon Title Integration Readiness

Objective

Four new titles are queued for Q3–Q4 2026. The manifold must be ready to absorb them without coupling increase. Each new game must slot into the architecture as cleanly as the first five.

AI Implementation Rules

Per-Title Substrate Skeleton:

- A directory with the game name
- A single `index` module that imports ONLY from the shared layer
- A coordinate namespace reservation (see table below)
- A lens template with domain declarations

Namespace Reservations:

Title	Namespace Prefix	Estimated Modules	Complexity Notes
Schwartz Diamond Connect Four	`schwartz.*`	~40	3D lattice physics, camera tracking, AI opponent
Title 2 (TBD)	`maze.*`	~20	Estimated based on mid-complexity reference

Title	Namespace Prefix	Estimated Modules	Complexity Notes
Title 3 (TBD)	`puzzle.*`	~15	Estimated based on Assemble-class complexity
Title 4 (TBD)	`runner.*`	~20	Estimated based on mid-complexity reference

Pre-Validation Requirements:

- Adding 4 new substrates does NOT increase shared layer size by more than 10%. If new functionality would exceed this, it belongs in the game substrate, not shared.
- Adding 4 new substrates does NOT add any new cross-substrate edges (Theorem 1 compliance)
- Runtime cost remains O(N·d) (Theorem 2 compliance)

Integration Test Templates (per new title):

19. New game substrate loads without side effects on existing games
20. New game's lenses read only declared coordinates
21. No naming collisions in the coordinate namespace
22. Shared layer imports are read-only — no writes back to shared state

Section 4: AI Collaboration Rules

These rules govern how any AI assistant should behave when helping Kenneth work on KensGames.com or the ButterflyFX™ engine. These are non-negotiable operational constraints.

Rule 4.1 — Respect the Manifold Thesis

- **Never** suggest architectural patterns that introduce coupling: no event buses, no shared mutable state, no pub/sub, no dependency injection containers, no service locators
- If Kenneth asks for a feature, the AI must **first** determine which substrate it belongs to and which coordinates it reads/writes
- The answer to “Where does this code go?” is always: *“Which point on the manifold surface does this occupy?”*

Rule 4.2 — Zero-Dependency Discipline

- **Never** suggest adding npm packages, frameworks, or external libraries unless Kenneth explicitly requests one
- The entire portal is hand-written JavaScript, WebGL2, and GLSL. All suggestions must work within this stack.
- If a feature would typically require a library (e.g., routing → react-router), provide a **vanilla implementation** that fits the existing architecture

Rule 4.3 — Formal Proof Awareness

- Any architectural suggestion must be checked against all three theorems
- If a suggestion would increase coupling complexity beyond O(N), flag it: **“⚠ Theorem 1 violation”**
- If a suggestion requires runtime coordination beyond O(N·d), flag it: **“⚠ Theorem 2 violation”**
- Always frame quantitative claims with the same intellectual honesty shown in Theorem 3’s disclaimer

Rule 4.4 — Brand Voice Consistency

DO	DO NOT
Technically precise language	Hype, buzzwords, marketing fluff
Quiet confidence	Exclamation marks, emojis in UI copy
Arcade cabinet instruction panel tone	Casual “Hey there!” or “Awesome!” voice
`"Connection lost. Reconnecting..."`	`"Oops! Something went wrong"`
Neon-on-dark aesthetic	Light themes, rounded cards, Material Design, shadows

Rule 4.5 — Performance as Architecture

Every suggestion must include its performance impact estimate, framed in these terms:

Metric	Budget Ceiling
Desktop frame rate	60 fps
Mobile frame rate	30 fps
Input latency	< 100 ms
Total bundle size	< 5 MB
Additional coordinate reads per frame	Must be stated
Additional GPU draw calls	Must be stated
Additional network roundtrips	Must be stated

Rule 4.6 — Founder-Linked Identity

- Kenneth Bingham is credited as the **sole creator**. AI-generated content for the site must not dilute this attribution.
- Suggestions for “About” or “Credits” sections must maintain the founder-linked narrative
- Never suggest team/company language — this is a solo paradigm project

Rule 4.7 — Ethical Positioning Guard

“No ads. No tracking. No pay-to-win.” is inviolable.

NEVER Suggest	Acceptable Revenue Channels
Advertising of any kind	Art Store (Shopify/Printful — already exists)
Data collection or sale	Steam / Google Play game sales
Loot boxes or gacha mechanics	Voluntary Substack subscriptions
Pay-to-win mechanics	Premium game content (no pay-to-win)
Third-party tracking scripts	Privacy-preserving analytics per Directive 2.6

Rule 4.8 — Coordinate-First Problem Solving

- When Kenneth describes a bug or feature, the AI should first ask: *“What coordinate on the manifold is affected?”*
- Debugging should start from the coordinate, trace through the lenses that read it, and identify which substrate writes it
- This is the **manifold-native debugging methodology** — it replaces the conventional “trace the event chain” approach

Debugging Flow

Step 1: Identify the coordinate → `game.score.current`

Step 2: List all lenses reading that coordinate → `scoreLens`, `highScoreLens`

Step 3: Identify the substrate that writes it → `fasttrack/scoring.js`

Step 4: Trace the write path, not an event chain

Section 5: Implementation Priority Matrix

Ordered from highest priority to lowest. Priority score is calculated as Impact × inverse Effort (High=3, Medium=2, Low=1).

#	Directive / Protocol	Impact	Effort	Priority Score	Sprint	Dependencies
1	**Deep-Link URL Routing (2.1)**	High	Medium	3 × 2 = 6	Sprint 1	None
2	**Mobile Responsiveness (2.2)**	High	High	3 × 1 = 3	Sprint 1–2	None
3	**Substrate Isolation (3.1)**	High	Medium	3 × 2 = 6	Sprint 1	None
4	**Zero-Coupling Enforcement (3.4)**	High	Low	3 × 3 = 9	Sprint 1	Depends on 3.1
5	**Casual Fast-Path (2.3)**	Medium	Low	2 × 3 = 6	Sprint 2	None
6	**Lens Composition Hardening (3.2)**	High	High	3 × 1 = 3	Sprint 2	Depends on 3.1
7	**Coordinate Density (3.3)**	Medium	Medium	2 × 2 = 4	Sprint 2–3	Depends on 3.1, 3.2

#	Directive / Protocol	Impact	Effort	Priority Score	Sprint	Dependencies
8	**Social Proof Integration (2.4)**	Medium	Medium	2 × 2 = 4	Sprint 3	None
9	**Shared Layer Audit (3.6)**	Medium	Medium	2 × 2 = 4	Sprint 3	Depends on 3.1
10	**Art Store Prominence (2.5)**	Low	Low	1 × 3 = 3	Sprint 3	None
11	**Privacy Analytics (2.6)**	Medium	Medium	2 × 2 = 4	Sprint 3	None
12	**Formal Proof Hardening (3.5)**	High	High	3 × 1 = 3	Sprint 4	Depends on 3.1–3.4
13	**Shader Substrate Alignment (3.7)**	Medium	Medium	2 × 2 = 4	Sprint 4	None
14	**Coming Soon Readiness (3.8)**	High	Medium	3 × 2 = 6	Sprint 4–5	Depends on 3.1–3.6

Sprint Timeline Overview

Sprint	Focus	Items
Sprint 1	Foundation — SEO + Manifold Integrity	2.1, 2.2 (start), 3.1, 3.4
Sprint 2	UX Polish + Lens Architecture	2.2 (complete), 2.3, 3.2, 3.3 (start)
Sprint 3	Community +	2.4, 2.5, 2.6, 3.3

Sprint	Focus	Items
	Optimization	(complete), 3.6
Sprint 4	Proofs + Shaders	3.5, 3.7, 3.8 (start)
Sprint 5	New Title Integration	3.8 (complete), full regression testing

Section 6: Validation and Acceptance Criteria

Every directive and protocol has specific, measurable acceptance criteria. A directive is not complete until all criteria are met.

Area	Acceptance Criterion	Measurement
SEO Routing (2.1)	All 11 canonical routes indexed in Google Search Console	Within 30 days of deployment
Mobile (2.2)	Lighthouse mobile performance score ≥ 85 on all routes	Lighthouse CI on each deploy
Mobile (2.2)	No horizontal overflow on any element at 320px viewport width	Automated viewport testing
Fast-Path (2.3)	PLAY NOW button visible within 3 seconds; scroll to game cards in < 500ms	Manual + automated UI test
Social Proof (2.4)	Player counter updates within 2 seconds of WebSocket connect/disconnect	WebSocket event timing
Art Store (2.5)	Art Store section visible in main content flow; all links open Shopify in new tab	Manual verification
Analytics (2.6)	Zero cookies set; zero PII collected; admin	Cookie audit + code review

Area	Acceptance Criterion	Measurement
	dashboard accessible	
Substrate Isolation (3.1)	Zero violations in automated build-time scan across all 185 modules	CI build gate
Lens Purity (3.2)	100% of lenses pass Proxy-based mutation detection in debug mode	Debug mode test suite
Coordinate Density (3.3)	`density_ratio` ≥ 0.85 (no more than 15% dead coordinates)	Build-time density report
Zero Coupling (3.4)	Zero event bus patterns detected by automated scanner	CI build gate
Formal Proofs (3.5)	Empirical measurements within 10% of theoretical predictions for Theorems 1–2	Instrumented game loop data
Shared Layer (3.6)	All shared modules used by ≥ 3 game substrates; zero dead modules	Usage count report
Shaders (3.7)	All shaders documented in `shaders.json`; no cross-domain uniform reads	Manifest validation script
Integration (3.8)	4 skeleton substrates created; each passes isolation scan; zero namespace collisions	Integration test suite
Performance	60fps desktop, 30fps mobile on all 5 game titles	Frame time instrumentation
Brand Voice	All user-facing text passes voice consistency review (no hype, no buzzwords)	Manual review by Kenneth

Section 7: Appendix — Quick Reference Cards

Compact reference cards for any AI assistant working on this project. These cards should be the first thing consulted when making architectural decisions.

Card A — Substrate Boundary Rules

SUBSTRATE BOUNDARY RULES

Rule

Detail

Game → Shared

PERMITTED

(read only)

Shared → Game

VIOLATION

—

never allowed

Game → Game

VIOLATION

—

always a coupling defect

Inter-module communication

Through coordinates only (CMC) — never direct calls or events

Card B — Coordinate Naming Convention

COORDINATE NAMING CONVENTION

Format: `{domain}.{entity}.{property}[.{subproperty}]`

Layer

Allowed Values

Domains

player

,

game

,

physics

,

ui

,

network

,

audio

,

input

Entities

Lowercase singular nouns:

ball

,

paddle

,

score

,

state

Properties

Lowercase descriptors:

position

,

velocity

,

current

,

phase

Reserved Prefixes

schwartz.*

,

maze.*

,

puzzle.*

,

runner.*

Examples: `player.identity.avatar` ✅ `game.state.phase` ✅ `physics.ball.velocity.x` ✅ `playerScore` ❌ `isGameOver` ❌

Card C — Lens Declaration Template

LENS DECLARATION TEMPLATE

/** * @lens scoreLens * @domain game.score * @pure true * @param {Object} substrate - The game substrate (read-only) * @returns {number} Current score value */ function scoreLens(substrate) { return substrate.game.score.current; } // Composition example: const composedLens = composeLens(displayLens, scoreLens); // composedLens.domain = union of both domains

Card D — Build Verification Checklist

BUILD VERIFICATION CHECKLIST

☐

Check

Gate

Fail Action

☐

Substrate isolation scan

PASS / FAIL

Block build

☐

Event bus detection

CLEAN / FOUND

Block build

☐

Dead coordinate scan

CLEAN / FOUND

Warning (block if density < 0.85)

☐

Lens domain verification

PASS / FAIL

Block build

☐

Bundle size ≤ 5 MB

PASS / FAIL

Block build

☐

Shared layer health: all modules used by ≥ 3 games

PASS / WARN

Warning + report

☐

Shader manifest: all uniforms documented

PASS / FAIL

Block build

☐

Route metadata: all canonical routes have unique SEO tags

PASS / FAIL

Block build

This document is the authoritative reference for all AI-assisted work on KensGames.com and the ButterflyFX™ engine. Any AI assistant receiving this document should treat its directives as binding constraints, not suggestions.

“The manifold is the architecture. The architecture is the product. Store geometry — not bytes.”

Kenneth Bingham

Sole Creator — KensGames.com & ButterflyFX™

20 April 2026

— END OF DOCUMENT —